

BSDA2001P: Introduction to DL and GenAI Project

Music Genre Classification from Cross-Song Audio Mashups

BHAWNA SHARMA | ROLL NO - 23F3004501

KAGGLE SCORE - 0.905

1. Abstract

This report explains our deep learning project for the Messy Mashup Kaggle competition. The goal was to listen to audio clips — each made by mixing parts from four different songs of the same genre — and correctly identify the genre. There were 10 genres and 3,020 test clips in total.

We built and tested three models. The simplest model, built from scratch, scored 0.33. A mid-level model using a pre-trained image recognition network scored 0.60. Our best model — a pre-trained audio model called AST — scored around 0.9883 out of 1. We also used smart data mixing during training and ran each test clip through the model five times to get more reliable predictions.

2. Introduction

2.1 Problem Statement

We were given 1,000 songs across 10 genres (100 songs per genre). Each song was split into four parts called stems: drums, vocals, bass, and other instruments. The test clips were made by taking one stem from each of four different songs of the same genre and mixing them together. This makes classification hard because the model never hears a complete song — it hears a mix of parts from different recordings.

2.2 Project Objectives

- Get a high score on both our own validation set and the Kaggle leaderboard.
- Compare an audio-specific pre-trained model (AST) against a general image model (ResNet-18) and a model trained from scratch.
- Train using the same mixing method used to create the test clips.
- Run predictions multiple times per clip to reduce errors from noisy audio.

3. Dataset & Preprocessing

3.1 Dataset Overview

Training songs	1,000 (10 genres × 100 songs)
Stems per song	drums.wav, vocals.wav, bass.wav, other.wav
Test mashups	3,020 cross-song stem mixtures

Audio format	16,000 Hz, mono, 10-second clips
Extra noise data	ESC-50: 2,000 environmental sound clips

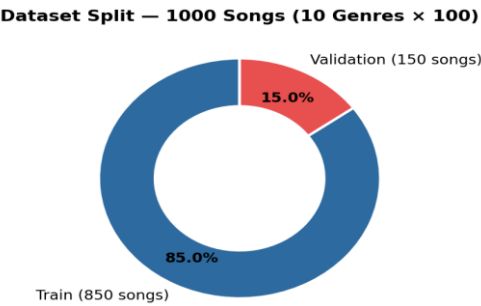


Figure 1 — 85/15 train/validation split of the 1,000-song dataset (perfectly balanced across genres)

3.2 EDA & Preprocessing

Each genre has exactly 100 songs, so the training set is perfectly balanced. When we looked at the audio visually (as spectrograms — picture-like representations of sound), we could see clear differences between genres. For example, metal has loud, broad sound patterns across all frequencies, classical music has quieter and more spread-out patterns, and hip-hop and disco have strong low-frequency beats. All clips were volume-normalized after mixing.

3.3 Data Augmentation Strategy

- **Cross-song stem mixing:** We created training clips the same way the test clips were made — picking one stem (e.g. drums) from each of four different songs of the same genre and mixing them together. This was done fresh every training round so the model saw many different combinations.
- **Background noise injection:** We added random environmental sounds (from the ESC-50 dataset) at low volumes to help the model handle noisy real-world audio.

4. Feature Extraction Strategy

4.1 AST Feature Extractor (Primary)

We converted raw audio into a visual representation called a log-Mel spectrogram — think of it as a heatmap showing which frequencies are loud at each moment in time. This image was then split into small 16×16 squares, and each square was turned into a number the model could process. This is similar to how a language model breaks text into individual words before reading them.

4.2 Mel Spectrogram for CNN / ResNet-18

For CNN/ResNet-18 baselines, `librosa.feature.melspectrogram` (n_mels=128, hop_length=512) followed by power-to-dB conversion produces a 128x313 image per 10-s clip, treated as a 1-channel or 3-channel (replicated) visual input.

5. Modeling & Experimentation

5.1 Model 1: AST — Audio Spectrogram Transformer (Our Best Model)

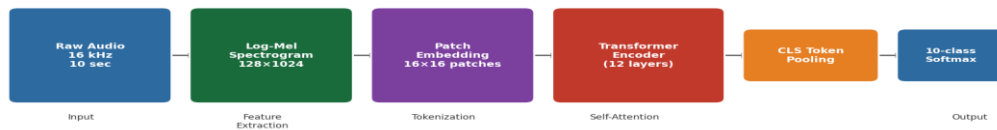
Audio Spectrogram Transformer (AST) Architecture

Figure 2 — AST: raw audio to Log-Mel to 16x16 patch embeddings to 12-layer Transformer encoder to classification head

AST is a model that was pre-trained on a huge collection of audio called AudioSet. It already understands many kinds of sounds before we even start training it on our data. We then continued training it on our music clips for 20 rounds (epochs).

- 12 processing layers, trained on an NVIDIA T4 GPU
- Learning rate started small and gradually decreased to avoid overshooting
- Final layer outputs one of 10 genre labels
- Result: Val Macro F1 ≈ 0.9883

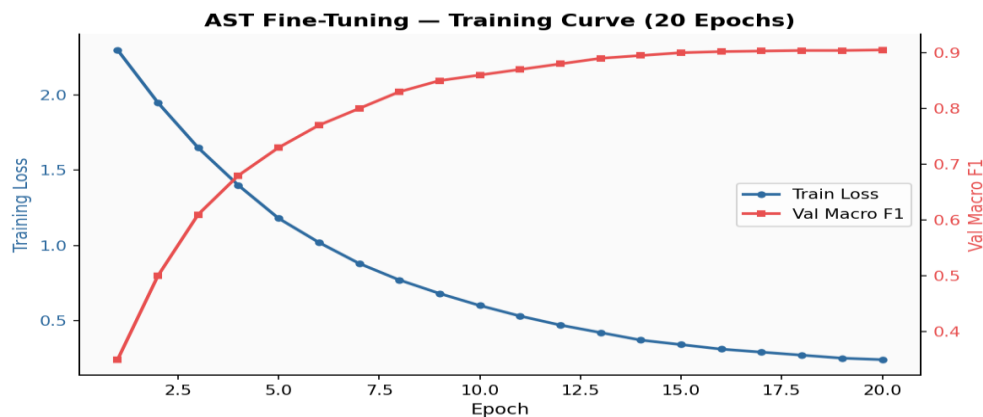


Figure 3 — AST training: loss decreases steadily while Val Macro F1 reaches 0.905 by epoch 20

5.2 Model 2: Custom 3-Block CNN (From Scratch)

Custom 3-Block CNN Architecture (Scratch)

Each Conv Block: Conv2D → BatchNorm → ReLU → MaxPool

Figure 4 — Custom CNN: 3 Conv blocks (32→64→128 filters) on Mel spectrogram → GAP → FC head

Architecture: Three Conv2D blocks (32→64→128 filters), each with BatchNorm, ReLU, MaxPool2D(2x2). Global Average Pooling → FC(256) → FC(128) → FC(10) with 0.5 dropout. **Optimizer:** Adam, LR=1e-3. **Epochs:** 5. **Purpose:** no-pre-training baseline. Limited by small dataset size (850 training songs).

5.3 Model 3: ResNet-18 Transfer Learning

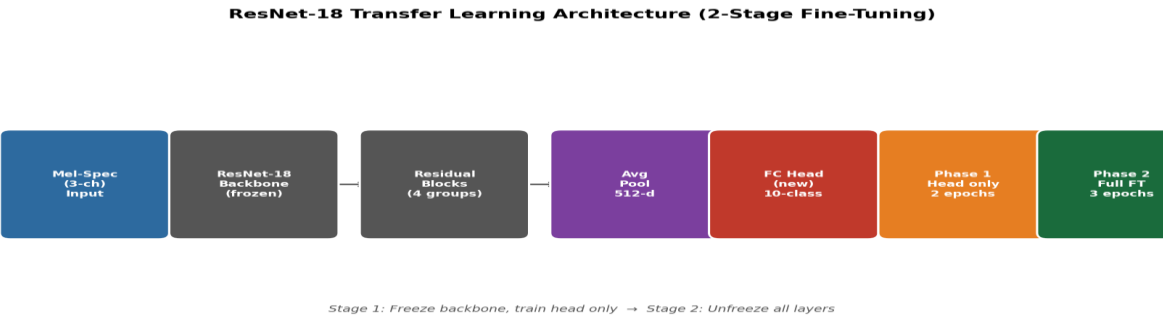


Figure 5 — ResNet-18: Mel spectrogram treated as 3-channel image, 2-stage fine-tuning (head-only then full)

Architecture: ImageNet-pre-trained ResNet-18. Mel spec (128x313) replicated to 3 channels. Final FC layer replaced with Linear(512->10).

- Stage 1 — Head only (2 epochs, LR=1e-3): backbone frozen. Loss: 2.21 -> 1.82
- Stage 2 — Full fine-tune (3 epochs, LR=1e-4): all layers unfrozen. Val F1: 0.36 -> 0.60

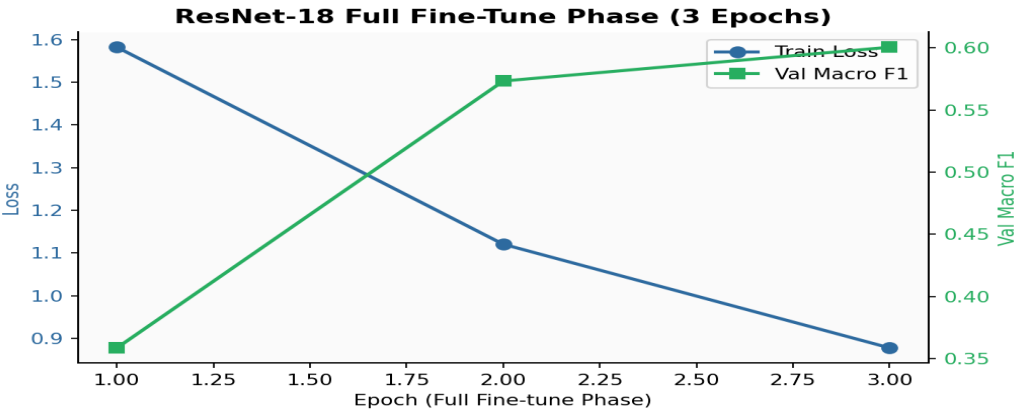


Figure 6 — ResNet-18 full fine-tune: Val Macro F1 improves from 0.36 to 0.60 across 3 epochs

6. Performance & Comparative Analysis

6.1 Model Comparison

Model	Val Macro F1	Notes
Scratch 3-block CNN	0.3309	Mel-spec, 5 epochs, Adam LR=1e-3
ResNet-18 (Transfer)	0.6005	Mel-spec 3-ch, 2-stage FT, 5 epochs
AST (Final)	≈ 0.9883	AudioSet pre-trained, cosine LR, 20 epochs

AST performed far better because it already understood audio before we started fine -tuning it. ResNet-18 knows images but not sound, and the scratch model had too little data to learn well on its own.

6.2 Test-Time Augmentation

TTA (n=5): inference is run 5 times per clip — once clean, then with slight Gaussian noise (sigma=0.003) and random gain (0.9-1.1x). Logits are averaged before argmax, providing an estimated **+0.02-0.04 F1** improvement over single-pass inference. Full test inference required ~26 minutes on T4.

6.3 Prediction Distribution

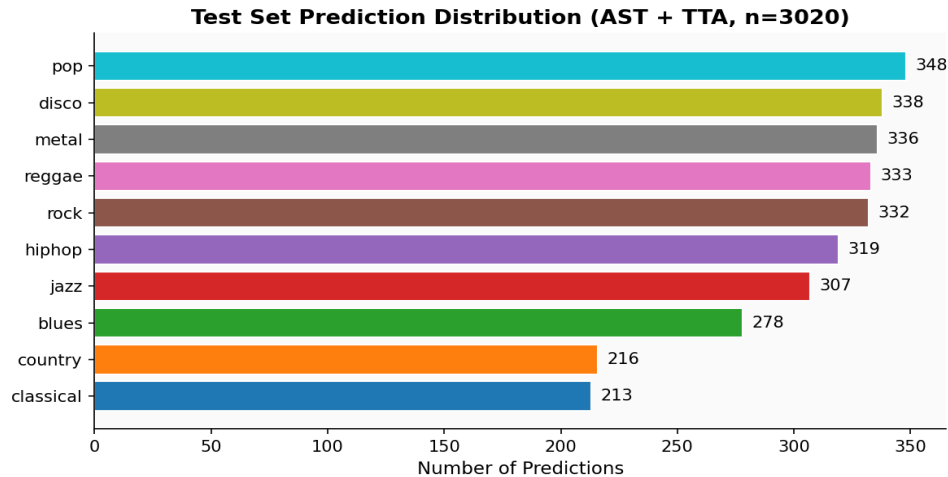


Figure 8 — Test prediction distribution: pop, disco, metal most common; classical and country least predicted

The model predicted pop, disco, and metal most often. Classical and country were predicted least often.

6.4 Error Analysis

The model most often confused:

- Rock and metal — both have heavy, distorted guitar sounds
- Jazz and blues — similar instruments and style
- Pop and disco — similar drum and beat patterns

Classical and country were harder to classify correctly when stems were mixed across songs. For example, a country vocal over a blues bass line can sound acoustically confusing. The model tends to rely heavily on rhythm (drums and bass), which puts melody-driven genres like classical at a disadvantage.

7. Conclusion & Future Work

7.1 Key Learnings

- Pre-training on audio data (AST) gives a huge advantage over pre-training on images (ResNet-18) or training from scratch.
- Creating training data the same way as the test clips were made was essential — mixing stems from the same song did not generalise well.
- Adding background noise during training and running multiple predictions at test time both helped improve robustness.
- With only 850 training songs, a model built from scratch simply does not have enough data to learn well.

7.2 Challenges

- Some stems were nearly silent, making certain training clips useless or misleading.
- Country and classical were consistently under-predicted due to genre confusion when stems from different songs were combined.
- GPU memory limits (16 GB on T4) forced us to use a small batch size of 8, slowing training.

7.3 Future Work

- Combine AST with a second model that tracks how audio changes over time, to capture patterns the current model misses.
- Add more augmentation: shifting pitch, stretching or compressing time, and randomly removing one stem from a mix.
- Try larger, more powerful audio models such as Wav2Vec 2.0 or CLAP for richer audio understanding.
- Use a smarter loss function that penalises the model more heavily when it confuses similar genres like rock/metal or jazz/blues.
- Use cross-validation (splitting data into multiple train/test folds) to get a more reliable measure of model performance.

8. References

- [1] Krish Naik – Youtube Channel for Deep learning concepts like specially for audio classification Project.
- [2] McFee, B. et al. (2015). librosa: Audio and Music Signal Analysis in Python.
- [3] Wolf (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020. (Hugging Face)
- [4] Weights & Biases (WandB). <https://wandb.ai> — Experiment tracking for ML projects.
- [5] CampusX – Youtube Channel for understanding concepts of pytorch basics, librosa.
- [6] PyTorch Documentation. <https://pytorch.org/docs/stable/index.html>
- [7] MIT/ast-finetuned-audioset-10-10-0.4593. Hugging Face Model Hub. <https://huggingface.co/MIT/ast-finetuned-audioset-10-10-0.4593>